PUCE · QUINTO SEMESTRE · DESARROLLO DE PLATAFORMAS

Funcionalidad y explicación del código

SecureGuard · centro de incidentes de seguridad

Semana 6 · autenticación, roles, validación y HTTPS

Qué explica este documento

Este documento recorre el proyecto SecureGuard de arriba abajo. Describe la funcionalidad completa, las herramientas, la base de datos, la seguridad, el papel de cada carpeta y de cada archivo propio del proyecto, y el trabajo de cada función del código de la aplicación.

- La interfaz web con la que una persona inicia sesión, consulta, registra, cambia el estado y elimina incidentes.
- La API HTTPS en Express: salud, login y el CRUD de incidentes.
- MySQL a través de Prisma 7, con las tablas Incidente y Usuario.
- JWT, bcrypt, roles, validación de entradas, HTTPS y el cuidado contra datos ejecutables en la pantalla.
- Las pruebas automáticas de validación y las capturas de la carpeta evidencias.

El alcance son los archivos que el proyecto escribió para funcionar. node_modules guarda librerías de terceros y .agents guarda guías de Prisma para el asistente. Esas dos carpetas no se ejecutan como lógica de SecureGuard y este documento no enumera sus funciones internas. Tampoco se copian contraseñas ni el valor de JWT_SECRET.

Capa	Dónde vive	Qué resuelve
Pantalla	public/	Login, listado, alta, filtro, botones según el rol y ficha estática.
API	server.js y src/	HTTPS, JWT, roles, validación y el CRUD de incidentes.
Datos	MySQL + Prisma	Tablas Incidente y Usuario. La contraseña se guarda como hash.

Alejandro Carriel

14 de octubre de 2026

Contenido

1. Qué hace el sistema . . . 3

2. Herramientas . . . 4

3. Base de datos . . . 5

3.1 Modelo Incidente . . . 5

3.2 Modelo Usuario . . . 6

3.3 Archivos de Prisma . . . 6

4. Seguridad . . . 6

4.1 HTTPS . . . 7

4.2 Contraseñas y token . . . 7

4.3 Roles . . . 7

4.4 Validación y salida . . . 7

4.5 Superficie publicada y CORS . . . 8

5. Recorrido de una petición . . . 8

6. Carpetas . . . 9

7. Archivos de arranque . . . 9

7.1 package.json . . . 9

7.2 server.js . . . 9

7.3 crear-cert.js . . . 10

7.4 prisma7.config.ts . . . 10

7.5 Entorno, Git y documentos de la raíz . . . 10

8. Código del servidor . . . 10

8.1 src/lib/prisma.js . . . 10

8.2 src/middleware/logger.js . . . 10

8.3 src/middleware/errorHandler.js . . . 11

8.4 src/security/hash.js . . . 11

8.5 src/security/token.js . . . 11
8.6 src/security/rol.js . . . 11
8.7 src/controllers/auth.controller.js . . . 11
8.8 src/controllers/incidentes.controller.js . . . 12
8.9 src/models/incidentes.model.js . . . 12
8.10 src/validation/incidentes.js . . . 13
8.11 src/routes/auth.routes.js . . . 14
8.12 src/routes/incidentes.routes.js . . . 14
8.13 src/seed.js . . . 14

9. Interfaz . . . 14

9.1 public/index.html . . . 14
9.2 public/detalle.html . . . 15
9.3 public/css/estilos.css . . . 15
9.4 public/js/modelo.js . . . 16
9.5 public/js/vista.js . . . 17
9.6 public/js/controlador.js . . . 17

10. Pruebas, evidencias y correcciones . . . 18

10.1 tests/incidentes-validation.test.js . . . 18
10.2 evidencias/ . . . 19

11. Comportamiento que el código deja fijo . . . 19

1. Qué hace el sistema

SecureGuard es un centro académico de respuesta a incidentes. Una persona del equipo entra con correo y contraseña, recibe un token y, según su rol, puede ver incidentes, crearlos, cambiar su estado, reemplazarlos o eliminarlos. Los datos viven en MySQL. El navegador habla con el servidor solo por HTTPS en el puerto 3000.

El producto tiene dos partes que se sirven juntas. El servidor Express, en la raíz del proyecto, expone la API y también entrega los archivos de public. El navegador abre https://localhost:3000, carga index.html y, desde ahí, el módulo controlador.js pide datos a /api.

Lo que una persona puede hacer en la pantalla

- **Entrar.** El formulario de acceso envía correo y contraseña. Si coinciden con un usuario activo, la interfaz guarda el token y muestra nombre y rol.

- **Salir.** El botón Salir borra la sesión del navegador y vacía el listado.
- **Ver incidentes.** Con sesión válida, la grilla muestra código, título, tipo, estado, fecha y severidad.
- **Filtrar.** El selector de severidad reduce las tarjetas en el navegador. El resumen de totales sigue contando todos los incidentes cargados.
- **Registrar.** El formulario pide título, descripción, severidad, tipo, fecha y correo, más una confirmación que no viaja al servidor.
- **Cambiar estado.** Supervisor y administrador ven un botón que pasa el incidente a Contención en curso.
- **Eliminar.** Solo el administrador ve el botón Eliminar.
- **Leer el caso destacado.** detalle.html muestra una ficha fija del caso INC-002. Esa página no consulta la API.

Lo que hace la API

Hay tres grupos de rutas. GET /api/salud responde sin token y confirma que el servicio está en HTTPS. POST /api/auth/login comprueba la cuenta y devuelve el token. Todo /api/incidentes exige un token válido de un usuario activo, y cada método además exige un rol.

Método y ruta	Quién puede	Resultado
`/api/salud`	Cualquiera	200. Confirma el servicio, la unidad 6 y el canal HTTPS.
`POST /api/auth/login`	Cualquiera	200 con token y datos públicos del usuario, o 401 si las credenciales fallan.
`GET /api/incidentes`	Cualquier rol con token	200 con el total y la lista. Acepta filtros por severidad, prioridad y estado.
`GET /api/incidentes/:id`	Cualquier rol con token	200 con un incidente, o 404 si no existe. El id puede ser el código INC-00N o el id numérico.
`POST /api/incidentes`	analista y admin	201 con el incidente creado. El estado queda En investigación.
`PATCH /api/incidentes/:id`	supervisor y admin	200. Solo acepta cambiar el campo estado.
`PUT /api/incidentes/:id`	admin	200. Reemplaza el conjunto completo de campos editables.
`DELETE /api/incidentes/:id`	admin	200 con los datos del incidente que se acaba de borrar, o 404.

Cuando falta el token, el token está mal o el usuario ya no está activo, la respuesta es 401. Cuando el token es bueno y el rol no alcanza, la respuesta es 403. Cuando el cuerpo no cumple las reglas, la respuesta es 400 e incluye un objeto detalles. Una ruta desconocida responde 404. Un fallo no previsto responde 500.

> **Dos cifras de la portada no salen de la base.** Las tarjetas Resueltos este mes (12) y Tiempo de respuesta (18m) están escritas fijas en index.html. Las cifras vivas son Incidentes registrados y Casos abiertos, y las calcula vista.js con la lista recibida.

2. Herramientas

El proyecto corre en Node.js. package.json declara el paquete secureguard-semana6, versión 1.0.0 y privado. El motor esperado, según el README, es Node.js 20 o superior, con MySQL o MariaDB en la máquina local.

Scripts de npm

Script	Comando	Para qué sirve
`start`	`node server.js`	Arranca el servidor HTTPS.
`cert`	`node crear-cert.js`	Crea el certificado y la llave de localhost.
`migrate`	`prisma migrate dev`	Aplica el esquema a la base. Usa prisma7.config.ts.
`generate`	`prisma generate`	Genera el cliente de Prisma a partir del esquema.
`seed`	`node src/seed.js`	Crea o actualiza las tres cuentas de prueba.
`test`	`node --test`	Ejecuta las pruebas del directorio tests con el corredor de Node.

Dependencias que sí usa el programa

Paquete	Papel en SecureGuard
`express 4`	Enruta la API, lee JSON y publica la carpeta public.
`@prisma/client 7 y prisma 7`	Cliente y CLI del ORM. El CLI está en devDependencies.
`@prisma/adapter-mariadb 7`	Adaptador que conecta Prisma 7 con MySQL o MariaDB.
`jsonwebtoken 9`	Firma y verifica el JWT con el secreto del entorno.
`bcryptjs 3`	Calcula y compara el hash de la contraseña. El costo es 10.
`cors 2`	Limita el origen, los métodos y los encabezados que el navegador puede usar.
`dotenv 17`	Carga DATABASE_URL, JWT_SECRET y PORT desde .env.
`selfsigned 5`	Genera el certificado autofirmado. Solo lo usa crear-cert.js.

Herramientas del navegador y de apoyo

- **Módulos ES del navegador.** index.html carga controlador.js con type=module. Ese archivo importa modelo.js y vista.js. No hay empaquetador.
- **sessionStorage.** Guarda token, rol y nombre mientras la pestaña sigue abierta.
- **Google Fonts.** DM Sans para el texto y Space Grotesk para los títulos.
- **jQuery 3.7.1.** index.html lo descarga desde code.jquery.com. Ningún archivo del proyecto llama a jQuery. Quedó como resto de la semana 2.
- **Unsplash.** estilos.css usa una foto remota como fondo del encabezado de la página principal.
- **node:test y node:assert/strict.** Librería estándar de Node para las pruebas. No hay Jest ni Mocha.
- **package-lock.json.** Fija las versiones exactas que instaló npm. No contiene lógica.

3. Base de datos

El motor es MySQL. Prisma declara provider = mysql y el archivo de bloqueo de migraciones repite ese proveedor. La conexión no está dentro de schema.prisma. En Prisma 7 la URL sale de prisma7.config.ts, que lee process.env.DATABASE_URL. El ejemplo del repositorio apunta a mysql://root:…@localhost:3306/incidentes. El nombre real de la base es el que tenga el archivo .env de cada máquina. Este documento no copia esa cadena.

src/lib/prisma.js construye un PrismaMariaDb con esa URL y un solo PrismaClient. El adaptador de MariaDB es el puente oficial de Prisma 7 para este tipo de servidor; también habla con MySQL. Todas las consultas del proyecto pasan por ese objeto exportado.

3.1 Modelo Incidente

Representa un caso de seguridad. La migración 20261007040417_init crea la tabla con utf8mb4 y collation unicode_ci. El cliente HTTP no recibe la fila completa: aCliente() esconde el id numérico y createdAt, y entrega el código en el campo id.

Campo	Tipo	Regla y uso
`id`	Int, clave, autoincremento	Identidad interna. La API también acepta este número al buscar, actualizar o borrar.
`codigo`	String, único	Código visible, por ejemplo INC-001. Lo genera siguienteCodigo().
`titulo`	String	Entre 5 y 100 caracteres después de recortar espacios.
`descripcion`	Text	Entre 20 y 500 caracteres. En MySQL es TEXT, no VARCHAR.
`severidad`	String	critica, alta, media o baja.

Campo	Tipo	Regla y uso
prioridad	String	Mismos cuatro valores. Si no llega, se copia la severidad.
estado	String	Valor inicial En investigación. También Contención en curso y Resuelto.
tipo	String	Texto de 2 a 80 caracteres. La pantalla ofrece phishing, malware, acceso y vulnerabilidad.
fecha	String	Fecha de calendario en texto AAAA-MM-DD. No es una columna DATETIME.
reportante	String	Correo o nombre de quien reporta. Al crear, se usa reportante, si no correo, si no el texto sin-correo.
createdAt	DateTime	Momento de inserción. Tiene default now() y no se envía al navegador.

3.2 Modelo Usuario

La migración 20261007044322_usuarios agrega la tabla de cuentas. El login busca por correo. La contraseña guardada es el hash de bcrypt, nunca el texto que la persona escribió.

Campo	Tipo	Regla y uso
id	Int, clave, autoincremento	Viaja dentro del JWT y sirve para volver a cargar la cuenta en cada petición.
nombre	String	Nombre visible. La interfaz lo guarda en la sesión.
correo	String, único	Identidad de login. Se compara en minúsculas y sin espacios laterales.
password	String	Hash bcrypt. No sale en la respuesta del login.
rol	String	analista, supervisor o admin. La autorización lee el rol de la base, no el rol guardado en el token.
activo	Boolean, default true	Si está en falso, el login y el token reciben 401.
createdAt	DateTime	Momento de alta de la cuenta. El programa no lo muestra.

3.3 Archivos de Prisma

- **prisma/schema.prisma.** Declara el generador prisma-client-js y los dos modelos. No trae la URL.
- **prisma7.config.ts.** Apunta al esquema, a prisma/migrations y a DATABASE_URL. Los scripts migrate y generate lo pasan con --config.
- **prisma/migrations/migration_lock.toml.** Fija el proveedor en mysql para que las migraciones no cambien de motor por accidente.
- **prisma/migrations/20261007040417_init/migration.sql.** CREATE TABLE de Incidente, índice único de codigo y clave primaria.
- **prisma/migrations/20261007044322_usuarios/migration.sql.** CREATE TABLE de Usuario, índice único de correo y clave primaria.
- **data/incidentes.json.** Tres incidentes de ejemplo de una etapa anterior. Ningún archivo .js ni .html lo importa. La ficha de detalle.html coincide con el segundo objeto, INC-002, pero está escrita a mano.

4. Seguridad

La seguridad está repartida entre el arranque del proceso, la carpeta security, la validación, la forma de pintar la interfaz y lo que el servidor se niega a publicar.

4.1 HTTPS

server.js no abre HTTP. Crea un servidor con el módulo https de Node y le entrega key.pem y cert.pem. Si uno de los dos archivos no existe, el proceso termina y pide npm run cert. crear-cert.js pide a selfsigned un certificado con nombre común localhost, vigencia de 365 días y llave de 2048 bits, y escribe la llave privada y el certificado en certs/. El navegador avisa porque la firma es propia, no de una autoridad pública. certs/ y cualquier archivo .pem o .key están en .gitignore.

4.2 Contraseñas y token

- **bcrypt.** crearHash() usa 10 rondas. coincide() compara el texto con el hash. El seed guarda solo el hash.
- **JWT.** firmar() mete el id y el rol, usa JWT_SECRET y vence a las 2 horas. El algoritmo por defecto de la librería es HS256.
- **Secreto fuerte.** Antes de escuchar el puerto, server.js exige que JWT_SECRET exista y que sea una cadena hexadecimal de 64 caracteres. Si no, el proceso sale con código 1 e indica cómo generar 32 bytes aleatorios.
- **Mensaje único de login.** Cuenta inexistente, cuenta inactiva y contraseña incorrecta responden lo mismo: Credenciales inválidas. El hash solo se compara cuando la cuenta existe y está activa.
- **Token en cada petición de incidentes.** exigirToken() lee Authorization, exige el prefijo Bearer y verifica la firma y la caducidad. Después vuelve a leer el usuario. Si no existe o no está activo, responde Usuario no activo.
- **El rol se confirma en la base.** exigirRol() mira req.usuario.rol, copiado de MySQL en esa petición. Un cambio de rol en la tabla aplica aunque el token todavía no venza. El rol dentro del token no decide el permiso.
- **Sesión del navegador.** El token queda en sessionStorage, no en una cookie. Al cerrar la pestaña desaparece. Un 401 de /incidentes borra la sesión y lleva al formulario de acceso. Un 403 explica el permiso y deja la sesión en su sitio.

4.3 Roles

Acción	analista	supervisor	admin
Ver listado y detalle	Sí	Sí	Sí
Crear incidente	Sí	No	Sí
Cambiar estado con PATCH	No	Sí	Sí
Reemplazar con PUT	No	No	Sí
Eliminar	No	No	Sí

La interfaz repite esa división en los botones: Cambiar estado aparece para supervisor y admin; Eliminar, solo para admin. El formulario de alta sigue visible para todos los roles. Si un analista lo envía, el servidor lo acepta. Si lo envía un supervisor, el servidor responde 403 y la pantalla muestra que el rol no alcanza. La interfaz no esconde el formulario según el rol; el permiso real está en la ruta.

4.4 Validación y salida

- **Lista blanca.** Crear, parchar y reemplazar rechazan cualquier propiedad que no esté prevista. id, codigo y campos inventados producen 400.
- **PATCH estrecho.** El único campo admitido es estado, y tiene que ser uno de los tres estados conocidos.
- **PUT completo.** Exige título, descripción, severidad, prioridad, estado, tipo y fecha. Si falta uno, rechaza el cuerpo.
- **Tipos y límites.** Textos recortados, longitudes mínimas y máximas, severidad de una lista, fecha real, no futura, y correo con forma básica cuando viene.
- **El modelo vuelve a filtrar.** actualizar() copia solo siete campos permitidos hacia Prisma, aunque la validación ya haya pasado.
- **Pantalla sin HTML de usuario.** pintarIncidentes() crea nodos y asigna textContent. La severidad solo se usa como clase si pertenece a critica, alta, media o baja. Así un título guardado no se ejecuta como marcado.

• **La confirmación no sale del navegador.** El checkbox confirmacion es obligatorio en el formulario y el controlador lo borra antes de armar el JSON.

4.5 Superficie publicada y CORS

express.static apunta únicamente a public/. Quien pida un archivo recibe index.html, detalle.html, el CSS o el JavaScript de esa carpeta. .env, certs/ y el código de src/ no están en ese directorio. CORS permite los orígenes https://localhost:3000, https://127.0.0.1:3000, http://127.0.0.1:5500 y http://localhost:5500. Los métodos permitidos son GET, POST, PUT, PATCH y DELETE. Los encabezados permitidos son Content-Type y Authorization. Esos dos orígenes en el puerto 5500 dejan que un Live Server de desarrollo hable con la API.

Qué queda fuera de estas medidas. El código no usa Helmet, no limita intentos de login y no emite un token de refresco. El cuerpo JSON usa el límite por defecto de Express. El certificado solo cubre un laboratorio local. Estas frases describen el programa tal como está, para no atribuirle controles que no contiene.

5. Recorrido de una petición

Alta de un incidente, de la pantalla a MySQL

• La persona envía el formulario. controlador.js corta el envío nativo, revisa título, descripción y fecha, y llama a reportValidity() para el resto de campos obligatorios.

• Quita confirmacion y llama a crearIncidente(). modelo.js hace POST https://localhost:3000/api/incidentes con Content-Type JSON y Authorization Bearer más el token.

• En el servidor pasan, en este orden, CORS, el logger, el lector JSON y el enrutador. La ruta de incidentes entra primero a exigirToken().

• exigirToken() verifica la firma y carga el usuario. exigirRol(analista, admin) compara el rol. Si falla, la respuesta se corta en 403.

• crear() del controlador llama a validarCreacion(). Si el cuerpo es válido, el modelo arma el código INC-00N, fija el estado En investigación y hace prisma.incidente.create.

• La respuesta 201 vuelve al navegador. El controlador vuelve a pedir la lista, vista.js pinta las tarjetas con textContent y anuncia que el incidente quedó en MySQL.

Orden interno de server.js

• Carga .env y comprueba JWT_SECRET, DATABASE_URL y los PEM.

• Aplica CORS, logger, express.json y el estático de public.

• Registra /api/salud, /api/auth y /api/incidentes.

• Cierra con noEncontrado y errorHandler, para que un next(error) de cualquier controlador caiga en el mismo formato JSON.

• Lee la llave y el certificado y escucha en PORT o, si no existe, en 3000.

El logger no lee el cuerpo ni el token. Cuando la respuesta termina, escribe en consola el método, la URL, el código HTTP y los milisegundos.

6. Carpetas

Carpeta	Qué guarda y qué hace
certs/	Llave privada y certificado de localhost. El servidor los lee al arrancar. Git los ignora.
data/	incidentes.json, muestra antigua de tres casos. El programa en ejecución no la lee.
evidencias/	Once capturas PNG de la API y de la interfaz. Sirven de prueba visual, no de código.
prisma/	Esquema, configuración de migraciones y los dos SQL de creación de tablas.
prisma/migrations/	Historial aplicado por prisma migrate. El lock fija el motor en mysql.
public/	Único directorio entregado al navegador: páginas, CSS y JavaScript de la interfaz.
public/css/	estilos.css, la hoja visual completa.
public/js/	modelo.js, vista.js y controlador.js. Separan datos, dibujo y eventos.
src/	Lógica de servidor: rutas, controladores, modelo, validación, seguridad, middleware, cliente Prisma y seed.
src/controllers/	Traducen la petición HTTP en una llamada al modelo o a Prisma y eligen el código de estado.
src/lib/	El único cliente de Prisma compartido.
src/middleware/	Registro de cada respuesta y formato común de error.
src/models/	Consultas de incidentes y la forma pública de cada fila.
src/routes/	Unen método, camino, rol y controlador.
src/security/	Hash, JWT y los dos middleware de acceso.
src/validation/	Reglas del cuerpo de incidentes, separadas del controlador.
tests/	Pruebas de esas reglas, sin levantar el servidor ni la base.
node_modules/	Paquetes instalados. No es código del proyecto.
.agents/	Guías locales de Prisma para el asistente. Git las ignora. La aplicación no las importa.

En la raíz también están server.js, crear-cert.js, package.json, package-lock.json, README.md, DM_CORRECCIONES.md, skills-lock.json, .gitignore, .env.example, .env y el informe previo T1S6_SecureGuard_Carriel.pdf. Cada uno se explica en las secciones siguientes. .env existe en el equipo de trabajo y Git no lo sigue.

7. Archivos de arranque

7.1 package.json

Identifica el proyecto, lista los seis scripts y separa dependencias de ejecución y de desarrollo. No tiene función propia. Es el contrato que usa npm para instalar y para lanzar start, cert, migrate, generate, seed y test.

7.2 server.js

Es el punto de entrada. No declara funciones con nombre. El trabajo está en el orden de ejecución del archivo y en el manejador anónimo de /api/salud.

comprobación de entorno

Si falta DATABASE_URL o JWT_SECRET, escribe el motivo y llama a process.exit(1). Si el secreto no cumple la expresión de 64 hexadecimales, también termina e indica el comando de crypto.randomBytes(32). Esta puerta corre antes de abrir el puerto, así que un secreto débil no llega a firmar tokens.

comprobación de certificados

Arma las rutas certs/key.pem y certs/cert.pem junto a server.js. Si falta alguno, termina con el aviso de ejecutar npm run cert.

`middleware y rutas`
Registra CORS con la lista de orígenes, el logger, el parser JSON y el estático de public. Monta la ruta de salud, el enrutador de auth en /api/auth y el de incidentes en /api/incidentes detrás de exigirToken. Al final monta el 404 y el manejador de errores.

`GET /api/salud`
Responde 200 con ok en true, el nombre SecureGuard API, la unidad 6 y el canal https. No mira el token. Sirve para saber que el proceso está vivo y que se entró por el servidor HTTPS.

`https.createServer(...).listen`
Lee los PEM de forma síncrona, crea el servidor TLS con la aplicación Express y escucha en PORT o en 3000. Al quedar listo, imprime la URL https://localhost y el puerto.

7.3 crear-cert.js

`main()`
Función asíncrona. Pide a selfsigned.generate un certificado cuyo commonName es localhost, con 365 días y keySize 2048. Crea la carpeta certs si no existe y escribe certs/key.pem con la llave privada y certs/cert.pem con el certificado. Luego imprime cert listo. El archivo llama a main() al cargarse. No recibe argumentos.

7.4 prisma7.config.ts

Módulo de configuración, sin funciones de negocio. Importa dotenv, llama a defineConfig y exporta el objeto por defecto. El objeto dice dónde está el esquema, dónde están las migraciones y que la URL es la variable DATABASE_URL. Prisma CLI lo usa; el servidor de Express no lo importa.

7.5 Entorno, Git y documentos de la raíz

- **.env.example.** Plantilla con DATABASE_URL de ejemplo hacia la base incidentes, un JWT_SECRET de marcador y PORT=3000. Incluye el comando para generar el secreto. No tiene valores reales.
- **.env.** Archivo local que leen dotenv y Prisma. Define la URL de MySQL, el secreto y el puerto. Está ignorado por Git y sus valores no se reproducen aquí.
- **.gitignore.** Excluye node_modules, .env, certs/, archivos .pem y .key, /src/generated/prisma, .agents/, .claude/, .windsurf/ y skills-lock.json.
- **README.md.** Explica el objetivo de la semana, la matriz de roles, la estructura, los comandos, los usuarios de prueba y las reglas de seguridad del repositorio.
- **DM_CORRECCIONES.md.** Bitácora de correcciones: listas blancas, campos que Prisma puede tocar, tarjetas sin innerHTML de datos, sesión ante 401, secreto hexadecimal, exclusión de confirmacion y las pruebas nuevas.
- **skills-lock.json.** Inventario de guías Prisma descargadas, con origen y hash. No participa en el arranque.
- **T1S6_SecureGuard_Carriel.pdf.** Informe de la entrega. Es un documento aparte de este archivo.

8. Código del servidor

8.1 src/lib/prisma.js

No define funciones. Vuelve a cargar dotenv, crea el adaptador PrismaMariaDb con DATABASE_URL, construye un PrismaClient con ese adaptador y lo exporta. auth.controller, el modelo de incidentes, el middleware de rol y el seed importan este mismo objeto, así que comparten una sola conexión lógica.

8.2 src/middleware/logger.js

`logger(req, res, next)`
Guarda la hora de entrada. Se suscribe a finish de la respuesta y, cuando esa respuesta ya se envió, imprime método, URL original, código de estado y la diferencia en milisegundos. Llama a next() enseguida, de modo que no espera al cierre para dejar seguir la cadena. Se exporta como función única del módulo.

8.3 src/middleware/errorHandler.js

`noEncontrado(_req, res)`
Responde 404 con ok en false y el mensaje Ruta no encontrada. Express lo alcanza cuando ningún manejador anterior respondió. El guion bajo del parámetro indica que la petición no se usa.

`errorHandler(error, _req, res, _next)`
Toma error.status o, si no viene, 500. Responde JSON con ok en false, error.message o el texto Error interno del servidor, y error.detalles cuando la validación lo agregó. Así un 400 de validación y un fallo inesperado salen con la misma forma. El cuarto parámetro tiene que existir para que Express reconozca este manejador como de error.

8.4 src/security/hash.js

`crearHash(clave)`
Devuelve la promesa de bcrypt.hash con la clave y 10 rondas de sal. La usa el seed al crear o actualizar cuentas. El login no vuelve a hashear para guardar; solo compara.

`coincide(clave, hash)`
Devuelve la promesa de bcrypt.compare. Es true cuando el texto corresponde al hash guardado. login() la espera con await y la niega dentro de la condición de rechazo.

8.5 src/security/token.js

`firmar(usuario)`
Llama a jwt.sign con el objeto { id, rol }, el secreto del entorno y expiresIn en 2h. Devuelve el token en texto. No incluye correo, nombre ni contraseña. login() lo pone en la respuesta.

`verificar(token)`
Llama a jwt.verify con el mismo secreto. Si la firma, el formato o la caducidad fallan, la librería lanza una excepción. exigirToken() atrapa esa excepción y responde 401 con Token inválido o vencido. Si sale bien, devuelve el payload, y el middleware usa payload.id para buscar la cuenta.

8.6 src/security/rol.js

`exigirToken(req, res, next)`
Lee el encabezado Authorization. Si empieza por Bearer y un espacio, se queda con el resto; si no, el token queda vacío y responde 401 Token ausente. Verifica el token, busca el usuario por payload.id y, si no hay fila o activo es falso, responde 401 Usuario no activo. Cuando todo cuadra, escribe req.usuario con id, correo y rol tomados de la base y llama a next(). Cualquier error de verificación cae en el catch y responde 401 sin distinguir el motivo interno.

`exigirRol(...roles)`
Es una fábrica. Recibe los roles permitidos y devuelve el middleware de esa ruta. Ese middleware compara req.usuario.rol con la lista. Si no está, responde 403 Permiso insuficiente. Si está, llama a next(). En el proyecto siempre corre después de exigirToken, que es quien crea req.usuario. GET no lo usa: ver incidentes le basta con tener un token de un usuario activo.

8.7 src/controllers/auth.controller.js

`login(req, res, next)`
Convierte el correo a texto, le quita espacios y lo pasa a minúsculas. La contraseña se convierte a texto, incluso si faltaba. Busca un usuario con findUnique por ese correo. Si no hay usuario, si no está activo o si coincide() devuelve falso, responde 401 Credenciales inválidas. Si pasa, responde 200 con ok, el token firmado y un usuario reducido a nombre, correo y rol. Un fallo de base u otro error inesperado se entrega a next para que lo formatee errorHandler.

8.8 src/controllers/incidentes.controller.js

Cinco funciones async comparten el mismo esquema: llaman al modelo, eligen 200, 201 o 404, y mandan cualquier excepción al manejador común. Ninguna lee el rol; eso ya ocurrió en la ruta.

`listar(req, res, next)`
Pasa a modelo.listar los query params severidad, prioridad y estado. Responde 200 con ok, total igual a la cantidad y datos con el arreglo ya convertido al formato del cliente.

`obtener(req, res, next)`
Pide modelo.obtener con req.params.id. Si el resultado es nulo, responde 404 Incidente no encontrado. Si existe, responde 200 con ese objeto en datos.

`crear(req, res, next)`
Pasa req.body por validarCreacion. Con el objeto ya limpio llama a modelo.crear y responde 201. La validación lanza un error con status 400, y por eso un cuerpo malo no llega a Prisma.

`actualizar(req, res, next)`
Sirve para PATCH y para PUT. Si el método es PATCH, valida con validarParche; en cualquier otro caso, con validarReemplazo. En la práctica el otro caso es PUT, porque la ruta no usa esta función para más métodos. Llama a modelo.actualizar con el id y los cambios. Un nulo produce 404. Un registro tocado produce 200.

`eliminar(req, res, next)`
Llama a modelo.eliminar. Si no había fila, responde 404. Si la había, responde 200 con los datos públicos de la fila que se eliminó.

8.9 src/models/incidentes.model.js

Este archivo es el único que escribe y lee la tabla Incidente. También decide qué campos ve el navegador y cómo se interpreta el id de la URL.

`aCliente(row)`
Si la fila es nula, devuelve null. Si existe, arma un objeto con id igual al codigo, más título, descripción, severidad, prioridad, fecha, estado, tipo y reportante. Deja fuera el id numérico, createdAt y cualquier otro campo interno. Todas las funciones de lectura y escritura pasan su resultado por aquí.

`donde(id)`
Construye el filtro de búsqueda. Siempre incluye codigo igual al id convertido a texto. Si Number(id) es un número distinto de cero, también acepta el id numérico, unidos con OR. Así INC-003 y 3 pueden encontrar la misma fila, y un código no numérico no intenta una comparación numérica.

`listar(filtro)`
Arma un where vacío y copia severidad, prioridad o estado solo cuando el filtro trae ese valor. Ejecuta findMany ordenado por id descendente, de modo que el alta más reciente queda primero, y mapea cada fila con aCliente.

`obtener(id)`
Hace findFirst con donde(id) y devuelve el resultado ya pasado por aCliente. Puede devolver null.

`siguienteCodigo()`
Busca la fila de mayor id. Calcula INC- seguido de ese id más uno, rellenado con ceros hasta tres dígitos. Si no hay filas, el número base es 0 y el primer código es INC-001. El valor se calcula en el momento de crear.

`crear(datos)`
Inserta un incidente. El código sale de siguienteCodigo(). Título y descripción se vuelven texto y se recortan. La severidad, el tipo y la fecha se copian. La prioridad es datos.prioridad o, si viene vacía, la severidad. El estado se fuerza a En investigación, aunque el cuerpo hubiera querido otro. El reportante es datos.reportante, si no datos.correo, si no el literal sin-correo. Devuelve la fila pública.

`actualizar(id, cambios)`
Busca la fila. Si no está, devuelve null. Recorre la lista fija titulo, descripcion, severidad, prioridad, estado, tipo y fecha, y copia únicamente las propiedades que el objeto cambios posee. Si llega severidad y no llega prioridad, iguala la prioridad a la severidad nueva. Actualiza por el id numérico real, no por el código, y devuelve la fila pública.

`eliminar(id)`
Busca la fila. Si no está, devuelve null. La borra con delete sobre el id numérico y devuelve aCliente de la fila tal como estaba antes del borrado. La respuesta permite a la interfaz saber qué caso desapareció.

8.10 src/validation/incidentes.js

Concentra las reglas del cuerpo. Exporta solo validarCreacion, validarParche y validarReemplazo. El resto son apoyos. Las constantes definen el contrato:

- **CAMPOS_CREACION:** titulo, descripcion, severidad, prioridad, tipo, fecha, correo y reportante.
- **CAMPOS_REEMPLAZO:** titulo, descripcion, severidad, prioridad, estado, tipo y fecha.
- **SEVERIDADES:** critica, alta, media y baja.
- **ESTADOS:** En investigación, Contención en curso y Resuelto.

`errorValidacion(detalles)`
Crea un Error con el mensaje Validación fallida, le pone status 400 y le adjunta detalles, que es un objeto campo a mensaje. errorHandler usa esas dos propiedades. La función devuelve el error; quien valida lo lanza.

`objetoValido(datos)`
Exige un objeto JSON. Rechaza null, un valor que no sea objeto y los arreglos. El detalle dice Se esperaba un objeto JSON.

`validarCampos(datos, permitidos)`
Compara las llaves recibidas con la lista blanca. Si sobra alguna, lanza 400 e incluye los nombres en el mensaje Campos no permitidos.

`validarTexto(datos, campo, opciones)`
Opciones por defecto: requerido, mínimo 1 y máximo 200. Si el campo no viene y no es requerido, no hace nada. Si el valor no es string, lanza. Recorta espacios y exige la longitud. Si pasa, reemplaza el valor original por el texto ya recortado. Así el resto del programa trabaja con el texto limpio.

`validarFecha(datos)`
Exige el patrón de cuatro dígitos, guion, dos, guion y dos. Construye una fecha UTC a medianoche y comprueba que el calendario no la haya movido, de modo que 2026-02-30 falla. También rechaza una fecha posterior al día UTC de hoy. Los mensajes distinguen formato, fecha imposible y fecha futura.

`validarSeveridad(datos, campo)`
Comprueba que el campo, ya sea severidad o prioridad, esté en SEVERIDADES. El mensaje pide seleccionar una severidad válida.

`validarTipo(datos)`
Reutiliza validarTexto sobre tipo, con mínimo 2 y máximo 80. No obliga a que el tipo sea una de las cuatro opciones del formulario; cualquier texto de esa longitud sirve.

`validarContenido(datos, opciones)`
Aplica título de 5 a 100, descripción de 20 a 500, severidad, tipo y fecha. Si viene prioridad, la trata como severidad. Si viene estado y no está en la lista, lanza. Si viene correo, limita a 254 caracteres y exige una forma con texto, arroba, texto, punto y texto. Si viene reportante, limita a 254. Cuando no hay prioridad y la llamada no es un reemplazo, copia la severidad en la prioridad. Devuelve el mismo objeto ya normalizado.

`validarCreacion(entrada)`
Comprueba el objeto, aplica la lista de creación y valida el contenido sobre una copia. estado no está permitido en un alta: si alguien lo manda, validarCampos lo rechaza. El estado inicial lo pone el modelo, no el cliente.

`validarParche(entrada)`
Solo admite la propiedad estado y exige que venga. Copia ese único campo, lo valida como texto de hasta 40 caracteres y comprueba que sea uno de los tres estados. Devuelve un objeto nuevo que no arrastra nada más del cuerpo original.

`validarReemplazo(entrada)`
Admite solo los campos de reemplazo y exige que estén los siete. Si falta alguno, el detalle lista los nombres. Llama a validarContenido con reemplazo en true, así que no inventa la prioridad: tiene que haber llegado. También valida el estado porque el objeto lo trae.

8.11 src/routes/auth.routes.js
Crea un Router de Express. La única ruta es POST /login, enlazada a login del controlador. Como server.js monta este router en /api/auth, la ruta pública queda POST /api/auth/login. No hay middleware de token: el login es la puerta de entrada. El módulo exporta el router.

8.12 src/routes/incidentes.routes.js
Crea el router que server.js monta en /api/incidentes, ya detrás de exigirToken. Cada línea agrega el rol cuando la acción no es de solo lectura.

- **GET /** llama a listar. Cualquier usuario activo.
- **GET /:id** llama a obtener. El parámetro se declara después de la ruta de colección, así que no captura la raíz.
- **POST /** pasa por exigirRol(analista, admin) y luego por crear.
- **PATCH /:id** pasa por exigirRol(supervisor, admin) y luego por actualizar.
- **PUT /:id** pasa por exigirRol(admin) y usa el mismo actualizar.
- **DELETE /:id** pasa por exigirRol(admin) y llama a eliminar.

8.13 src/seed.js
`main()`
Define tres cuentas locales: Ana Analista con analista@secureguard.edu, rol analista y clave analista123; Sol Supervisor con supervisor@secureguard.edu, rol supervisor y clave supervisor123; Ada Admin con admin@secureguard.edu, rol admin y clave admin123. Por cada una calcula el hash y hace upsert por correo. Si el correo ya existe, actualiza nombre, hash, rol y deja activo en true. Si no existe, la crea. Al terminar imprime Usuarios listos. El archivo llama a main() y, en finally, cierra Prisma con $disconnect, tanto si salió bien como si falló. Estas claves son de demostración en el código fuente y en el README. No son cuentas de un entorno real.

9. Interfaz
La interfaz sigue un reparto simple. modelo.js habla con la API y con sessionStorage. vista.js dibuja. controlador.js escucha el DOM y une las dos piezas. detalle.html es una página estática, fuera de ese ciclo.

9.1 public/index.html
Documento en español, con viewport para pantallas chicas y una descripción del panel. El título es SecureGuard | Centro de incidentes. Enlaza las fuentes, estilos.css y jQuery. Al final del cuerpo carga controlador.js como módulo, para que los import funcionen.

Bloques de la página

- **header.hero.** Franja de presentación, enlace interno al formulario y la pastilla Centro operativo. La foto la pone el CSS sobre .hero-media.
- **nav.site-nav.** Marca que vuelve a index.html y enlaces a Incidentes, Registrar y detalle.html.
- **#caja-login.** Formulario #formulario-login con #login-correo, #login-clave y el botón #cerrar-sesion. #sesion-actual empieza en Sin sesión.
- **#mensaje-estado.** Región aria-live, escondida visualmente con .sr-only, donde el controlador anuncia resultados a lectores de pantalla.
- **.metrics.** Cuatro tarjetas. #total-incidentes y #incidentes-abiertos los llena el script. Las otras dos cifras están fijas.
- **#listado-incidentes.** Select #filtro-severidad, párrafo #estado-carga y grilla vacía #lista-incidentes.
- **#registro-incidente.** Formulario #formulario-incidente. Los campos son #titulo, #descripcion, #severidad, #tipo, #fecha, #correo y el checkbox #confirmacion. Cada campo obligatorio trae minlength, maxlength o required, y varios tienen un texto de ayuda.
- **aside de guía SOC.** Texto fijo de tres pasos: identificar, aislar y documentar. No tiene lógica.
- **footer.** Nombre del proyecto, la materia y el crédito a Alejandro Carriel, 2026.

Los id que el JavaScript busca son los del login, la sesión, el aviso, el filtro, la lista, el formulario, el título, la descripción, la fecha y los dos contadores. Si uno de esos id cambiara, la función que lo usa dejaría de encontrar el nodo.

9.2 public/detalle.html

Página estática del caso destacado. Repite cabeza, navegación y pie, con una franja más baja gracias a .hero-compact. Marca el enlace actual con aria-current=page. La ficha está escrita en el HTML: código INC-002, tipo Acceso no autorizado, severidad Crítica, estado Contención en curso, fecha 2026-09-02 y reportante admin@secureguard.edu, más un párrafo de descripción. No carga JavaScript, no pide token y no lee MySQL. El botón vuelve al ancla del listado en index.html. El panel lateral recuerda que un caso crítico sigue abierto hasta tener responsable, evidencia y hora de contención.

9.3 public/css/estilos.css

Una sola hoja, sin preprocesador y sin funciones. El selector universal activa border-box. :root define la paleta: verdes #0f6b5c y #0b4f44, tinta #1c2430, texto secundario #5b6570, línea #d7ddd9, fondo #f4f6f4, blanco, y cuatro colores de severidad. El cuerpo usa DM Sans; los títulos, Space Grotesk.

Selectores	Qué dibujan
`.eyebrow`	Rótulo verde, pequeño y en versalitas, encima de los títulos.
`.hero, .hero-compact, .hero-media, .hero-content, .hero-status, .status-dot`	Portada oscura, foto con velo, texto encima y pastilla de estado con punto verde.
`.site-nav, .brand, .brand-mark`	Barra blanca, marca con la letra S en un cuadro verde y enlaces en fila.
`.button, .button-primary`	Botón en línea, sin borde. El primario es verde y se oscurece al pasar el cursor.
`main, .metrics, .metric-card, .metric-highlight`	Columna central de hasta 1120 px y cuatro métricas. La primera invierte el color.
`.content-section, .insight-panel, .section-heading`	Tarjetas blancas con borde. El panel lateral es verde con texto blanco.
`.incident-grid, .incident-card, .critica, .alta, .media, .baja, .badge, .card-meta`	Grilla de tres columnas. Cada tarjeta lleva una franja superior del color de su severidad.
`.workspace-grid, .form-grid, .campo, .campo-wide`	Formulario a dos columnas, con título y descripción ocupando el ancho completo, más el panel de guía.
`label, input, select, textarea y su :focus`	Campos al 100 %, borde gris y contorno verde al enfocar.

Selectores	Qué dibujan
`.checkbox, .tip-list, .detail-list, footer`	Confirmación en fila, pasos numerados, ficha de dos columnas en el detalle y pie repartido.
`@media (max-width: 800px)`	Pasa grillas, navegación, encabezados y pie a una columna, y acorta la portada.
`.sr-only`	Saca el aviso de la vista sin quitarlo de los lectores de pantalla.
`.field-message`	Mensaje de error de campo, en el rojo de la severidad crítica.

index.html usa las clases load-state y required-note. La hoja no les asigna reglas, así que se ven con el estilo general del párrafo y del texto. Eso no rompe la página: el estado de carga sigue siendo un párrafo normal y la nota de campos obligatorios sigue siendo un span.

9.4 public/js/modelo.js

La constante API vale https://localhost:3000/api. Las funciones que otros módulos usan están exportadas. token() y pedir() quedan privadas del módulo.

`token()`

Devuelve el token guardado en sessionStorage o una cadena vacía. No valida si expiró; eso lo decide el servidor.

`pedir(ruta, opciones)`

Hace fetch a API más la ruta. Fuerza Content-Type JSON y, si hay token, el encabezado Bearer. Mezcla otras opciones que le pasen, como method y body. Lee el cuerpo como JSON. Si el estado es 401 y la ruta empieza por /incidentes, llama a cerrarSesion, dispara el evento sesion-expirada y lanza un error que pide volver a entrar. Un 403 lanza el mensaje de permisos del rol y no toca la sesión. Cualquier otra respuesta no exitosa lanza cuerpo.mensaje o, si no viene, Error de la API. Si todo va bien, devuelve el cuerpo completo.

`guardarSesion(datos)`

Guarda datos.token, datos.usuario.rol y datos.usuario.nombre. No guarda el correo ni el objeto completo. La usa el envío del login.

`cerrarSesion()`

Ejecuta sessionStorage.clear(), así que borra token, rol, nombre y cualquier otra clave de esa pestaña.

`sesion()`

Devuelve un objeto con token, rol y nombre leídos del almacenamiento. Si falta rol o nombre, usa cadena vacía. controlador.js lo usa para el texto de la sesión y para decidir los botones de cada tarjeta.

`entrar(correo, password)`

POST a /auth/login con esas dos propiedades. Devuelve la promesa del cuerpo. Un 401 de login no dispara sesion-expirada, porque la ruta no empieza por /incidentes; el mensaje de credenciales llega por el error general.

`obtenerIncidentes()`

GET a /incidentes y devuelve solo cuerpo.datos, que es el arreglo. El total queda fuera.

`filtrarPorSeveridad(incidentes, severidad)`

Si la severidad está vacía, devuelve el mismo arreglo. Si tiene valor, devuelve los incidentes cuya severidad es exactamente esa. No llama al servidor. El filtro de la API existe, pero esta pantalla no lo usa.

`crearIncidente(datos)`

POST a /incidentes con el objeto ya armado por el controlador. Devuelve el cuerpo de la respuesta, que incluye el incidente creado.

`cambiarEstado(id, estado)`

PATCH a /incidentes/ más el id, con un JSON que solo contiene estado. En esta interfaz el controlador siempre le pasa Contención en curso.

eliminarIncidente(id)
DELETE a /incidentes/ más el id, sin cuerpo. El id que llega desde la tarjeta es el código visible, por ejemplo INC-004, y la API lo acepta.

9.5 public/js/vista.js

NOMBRES traduce los cuatro códigos de severidad a Crítica, Alta, Media y Baja. No se exporta.

pintarIncidentes(incidentes, contenedor, rol)
Si la lista está vacía, deja en el contenedor un párrafo: No hay incidentes con ese filtro. Si hay elementos, crea por cada uno un article. La clase de severidad solo se aplica cuando el valor está en NOMBRES; si no, usa media, para no aceptar una clase arbitraria. El código va en un span, la severidad visible en una insignia, el título en un h3 y el tipo junto al estado en un párrafo, todo con textContent. La fecha va en un time con dateTime. Para supervisor y admin agrega un botón Cambiar estado cuyo data-estado es el id público. Para admin agrega Eliminar con data-borrar. Al final replaceChildren sustituye de una vez el contenido anterior, así no quedan tarjetas viejas.

pintarResumen(incidentes)
Busca #total-incidentes y #incidentes-abiertos. Si falta alguno, sale sin hacer nada: por eso detalle.html puede convivir con la hoja sin estos nodos. Escribe la cantidad total y la cantidad cuyo estado no es Resuelto.

pintarEstado(texto)
Escribe el texto en #estado-carga, el párrafo que está sobre la grilla. Si el nodo no existe, no hace nada.

9.6 public/js/controlador.js

Importa las operaciones del modelo y las tres funciones de dibujo. Guarda en constantes la lista, el filtro, el formulario, el aviso accesible y el texto de sesión. La variable incidentes conserva la última lista completa traída del servidor.

anunciar(texto)
Lo escribe en #estado-carga y también en #mensaje-estado. Una persona ve el aviso sobre la grilla y un lector de pantalla recibe la misma frase por la región aria-live.

mostrar(datos)
Pinta las tarjetas con datos y con el rol actual. El resumen, en cambio, usa siempre el arreglo completo incidentes, no el subconjunto filtrado. Por eso elegir Alta cambia las tarjetas y deja quietos los contadores globales.

iniciar()
Lee la sesión. Si hay token, #sesion-actual muestra nombre y rol; si no, muestra Sin sesión, anuncia que hay que ingresar y termina sin llamar a la API. Con token, pide los incidentes, los guarda, los pinta y anuncia cuántos llegaron desde MySQL. Si la petición falla, anuncia el mensaje del error. Se llama al final del archivo y también después de entrar o salir.

envío de #formulario-login
preventDefault evita la recarga. Lee correo y password con FormData, espera a entrar(), guarda la sesión, limpia el formulario y llama a iniciar() para cargar los incidentes. Si el servidor rechaza las credenciales, anuncia ese mensaje y no guarda nada.

clic en #cerrar-sesion
Cierra la sesión, vacía el arreglo, limpia la grilla con innerHTML vacío y llama a iniciar(), que ahora ve que no hay token.

evento sesion-expirada
Lo dispara pedir() antes de lanzar el error de un 401. Vacía los datos y la grilla, pone Sin sesión y mueve el hash a #caja-login para dejar el formulario de acceso a la vista. El catch de la acción que estaba en curso anuncia, además, que la sesión venció.

`change de #filtro-severidad`
Vuelve a mostrar el resultado de filtrarPorSeveridad con el valor elegido. No vuelve a pedir la lista al servidor.

`marcarError(id, mensaje)`
Busca el campo por id. Si todavía no existe #id-error, crea un small con la clase field-message y lo inserta justo después del campo. Escribe el mensaje y llama a setCustomValidity, de modo que reportValidity() considere el campo inválido y el navegador lo señale.

`envío de #formulario-incidente`
Cancela el envío nativo y limpia la validez personalizada de inputs, selects y textareas. Recorta título y descripción, toma la fecha y el día UTC de hoy. Si el título tiene menos de 5 caracteres, la descripción menos de 20 o la fecha es futura, marca el campo. Si reportValidity() falla, anuncia que hay errores y no llama a la API. Si pasa, arma el objeto con FormData, borra confirmacion, crea el incidente, recarga la lista, la pinta, anuncia el título guardado y reinicia el formulario. Un 400 o un 403 del servidor se anuncian con el mensaje recibido.

`clic en #lista-incidentes`
Mira data-borrar y data-estado del elemento pulsado. Si hay borrar, llama a eliminarIncidente. Si hay estado, llama a cambiarEstado con ese id y el texto fijo Contención en curso. El botón no recorre los demás estados ni envía Resuelto. Cuando hubo alguna de las dos acciones, recarga la lista, la pinta y anuncia Incidente eliminado o Estado actualizado. Un clic en el título o en la tarjeta no tiene dataset y no hace nada.

La última línea del archivo llama a iniciar(). Por eso, al recargar la página, una sesión todavía viva en sessionStorage vuelve a traer los incidentes sin pedir de nuevo la contraseña, mientras el token siga siendo válido en el servidor.

10. Pruebas, evidencias y correcciones

10.1 tests/incidentes-validation.test.js

Usa el corredor node:test y assert.strict. Importa las tres funciones públicas de validación. No abre el puerto, no usa Prisma y no necesita .env. incidenteValido es un alta de ejemplo con título y espacios laterales, una descripción de más de 20 caracteres, severidad alta, tipo phishing y fecha 2026-09-03.

`errorDeValidacion(funcion, entrada)`
Ayuda de prueba. Afirma que la función lanza y que el error trae status 400. No revisa el texto del detalle, solo el código.

`validarCreacion normaliza texto y calcula la prioridad`
Comprueba que el título quede sin espacios laterales y que la prioridad ausente se llene con alta, igual a la severidad.

`validarCreacion rechaza propiedades ajenas`
Manda un alta válida más el campo estado y espera el 400. estado no pertenece al contrato de creación.

`validarCreacion rechaza severidad y fechas no válidas`
Prueba la severidad urgente, que no está en la lista, y la fecha 2026-02-30, que el calendario no puede representar.

`validarParche permite únicamente un estado conocido`
Acepta { estado: Resuelto }. Rechaza ese mismo estado acompañado de un título. Rechaza el estado Cerrado, que no está en la lista.

`validarReemplazo exige el conjunto y rechaza campos internos`
Acepta un objeto con los siete campos editables. Rechaza el mismo objeto si se le agrega id. Rechaza un cuerpo que solo trae el título, porque faltan los demás.

npm test ejecuta node --test, que descubre este archivo dentro de tests. El archivo no cubre login, roles, Prisma ni la interfaz. Esas partes quedaron demostradas con las capturas.

10.2 evidencias/

Archivo	Qué muestra
01-login.200.png	Un login correcto y la respuesta 200 con token.
02-sin-token-401.png	Una petición de incidentes sin token y el 401.
03-analista-delete-403.png	Un analista intenta borrar y recibe 403.
04-titulo-corto-400.png	La validación rechaza un título demasiado corto.
05-admin-crea-201.png	Un administrador crea un incidente y recibe 201.
06-supervisor-patch-200.png	Un supervisor cambia el estado y recibe 200.
07-token-falso-401.png	Un token inventado recibe 401.
08-despues-de-reiniciar.png	Los datos siguen en MySQL después de reiniciar el proceso.
09-login-pagina.png	La pantalla de ingreso.
10-analista-sin-borrar.png	La vista del analista, sin el botón de eliminar.
11-admin-con-botones.png	La vista del administrador, con las acciones de la tarjeta.

DM_CORRECCIONES.md relaciona esas capturas con los cambios de código: la lista blanca por método, la lista de campos en el modelo, el dibujo con textContent, el cierre de sesión ante 401, la regla del secreto, la exclusión de confirmacion y la aparición de este archivo de pruebas.

11. Comportamiento que el código deja fijo

Estas reglas no son fallos de lectura: son decisiones visibles en las funciones. Conviene tenerlas presentes al usar o al extender el programa.

- **El botón Cambiar estado tiene un solo destino.** cambiarEstado recibe siempre Contención en curso. No hay control en la pantalla para pasar un caso a Resuelto ni para devolverlo a En investigación. Esos estados sí existen en la validación y se pueden enviar con PATCH desde un cliente que arme otro JSON.
- **PUT no tiene botón.** La ruta existe y solo el admin puede usarla. La interfaz nunca llama a PUT.
- **El formulario de alta no se oculta por rol.** El supervisor lo ve. El rechazo llega como 403 después del envío.
- **El filtro es local.** La API filtra si se le mandan query params. La pantalla baja la lista completa y filtra por severidad en memoria.
- **El resumen ignora el filtro.** pintarResumen cuenta el arreglo completo guardado en iniciar o después de cada recarga.
- **La fecha se juzga en UTC.** Tanto validarFecha como el controlador comparan contra toISOString cortado a diez caracteres. Las dos puntas usan el mismo criterio.
- **El código visible depende del último id numérico.** siguienteCodigo suma uno al mayor id y lo formatea. No mira el mayor código existente. Si se borra la última fila y ese código ya existe, la restricción única de MySQL rechaza el alta. Ese error sale como 500, porque el modelo no lo traduce.
- **El alta no acepta estado.** crear() lo escribe fijo. Un cliente no puede nacer un incidente ya resuelto.
- **prioridad sigue a severidad** cuando el parche o la actualización traen severidad y omiten prioridad. En un PUT la prioridad es obligatoria, así que esa copia automática no se necesita.
- **detalle.html puede quedar desactualizado.** Su INC-002 es texto fijo. Si ese caso cambia o desaparece en MySQL, la página destacada no se entera.
- **data/incidentes.json no alimenta la aplicación.** Borrar o editar ese archivo no cambia lo que muestra el servidor.
- **jQuery viaja en la página y no se usa.** El comportamiento actual está en los tres módulos de public/js.
- **Dos métricas son decorado.** 12 resueltos y 18 minutos no se calculan.

• **La sesión dura lo que dure el token y la pestaña.** A las dos horas el servidor responde 401, pedir() limpia sessionStorage y la interfaz vuelve al acceso. Cerrar la pestaña también borra el token, aunque todavía no haya vencido.

• **Un usuario inactivo queda fuera en el siguiente request.** Aunque conserve un token bien firmado, exigirToken lee activo en la base.

Puesta en marcha, en el orden que el código espera. Instalar dependencias, copiar .env.example a .env, completar la URL de MySQL y un JWT_SECRET hexadecimal de 64 caracteres, ejecutar npm run cert, npm run migrate, npm run seed y npm start. La aplicación queda en https://localhost:3000. El navegador pedirá aceptar el certificado autofirmado. npm test puede correr aparte: solo ejercita la validación.

Con eso queda descrito el recorrido completo: la carpeta que el navegador ve, la que el servidor ejecuta, las dos tablas, cada ruta, cada función propia del proyecto y las reglas con las que SecureGuard acepta o rechaza una acción.